Seminar: Large Language Models

Mixture of Experts

Igor Jakus 14.10.2026

Knowledge vs compute

Parameters

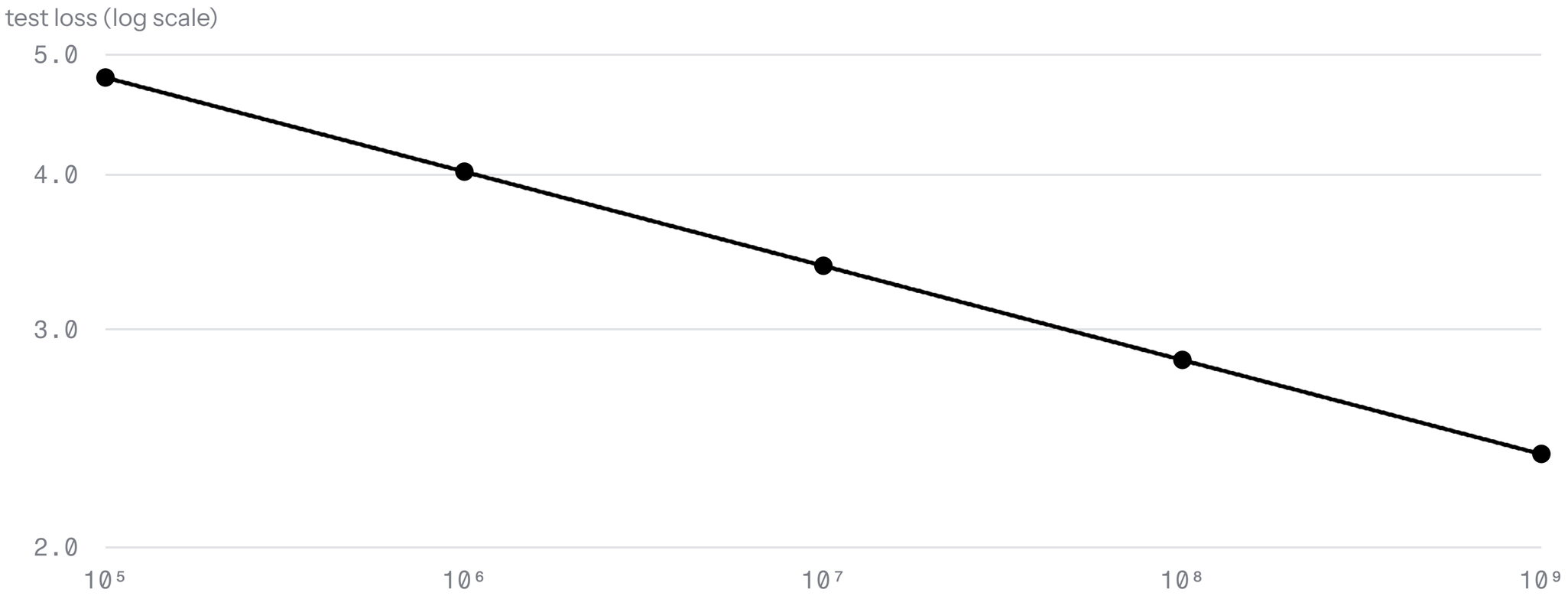
~how much the model can memorize

FLOPs per token

how much it costs to process each token

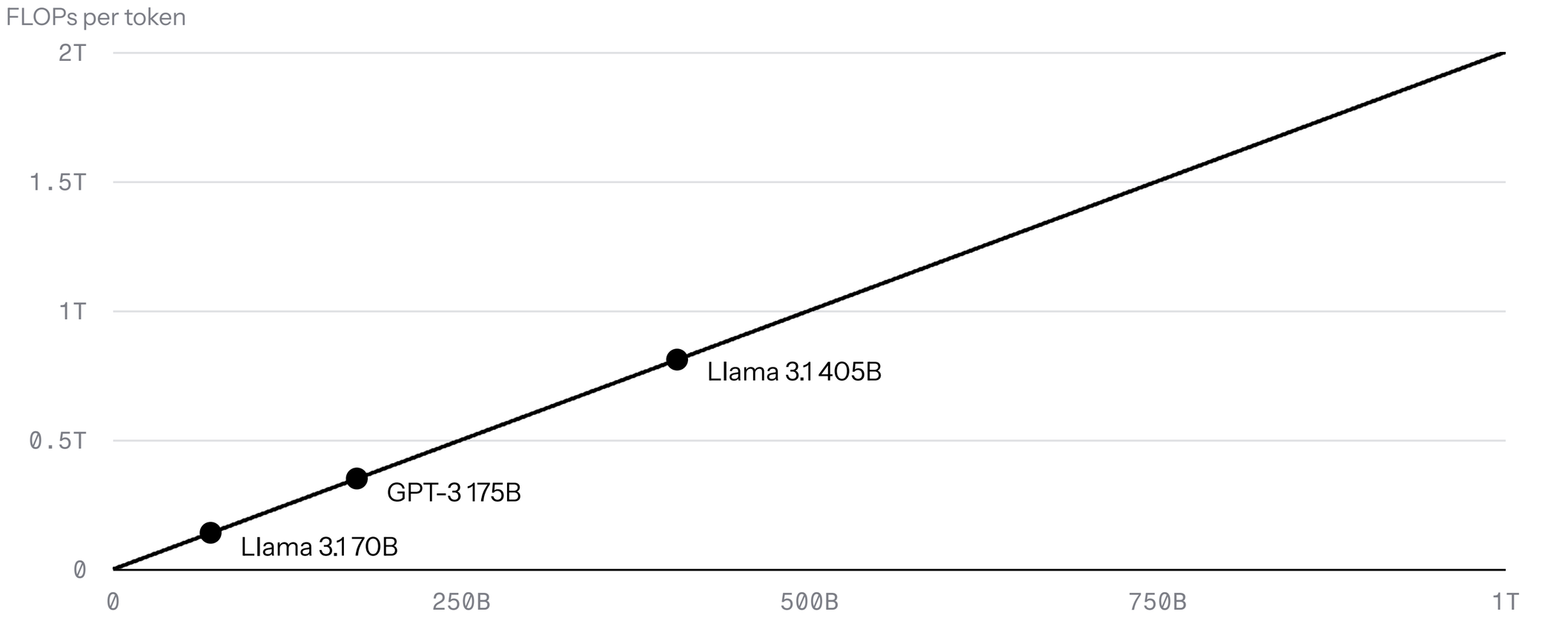
Scaling laws

more parameters → lower loss

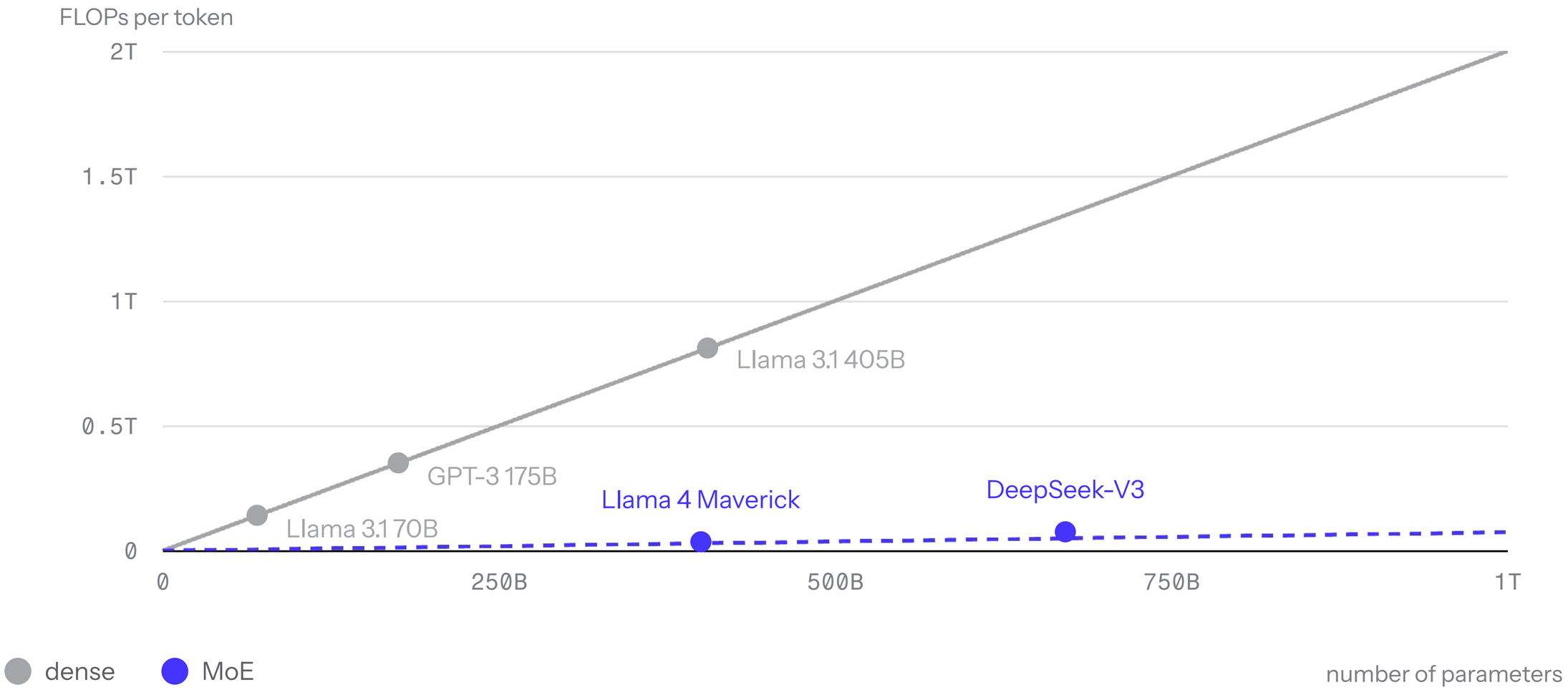


Kaplan et al., 2020

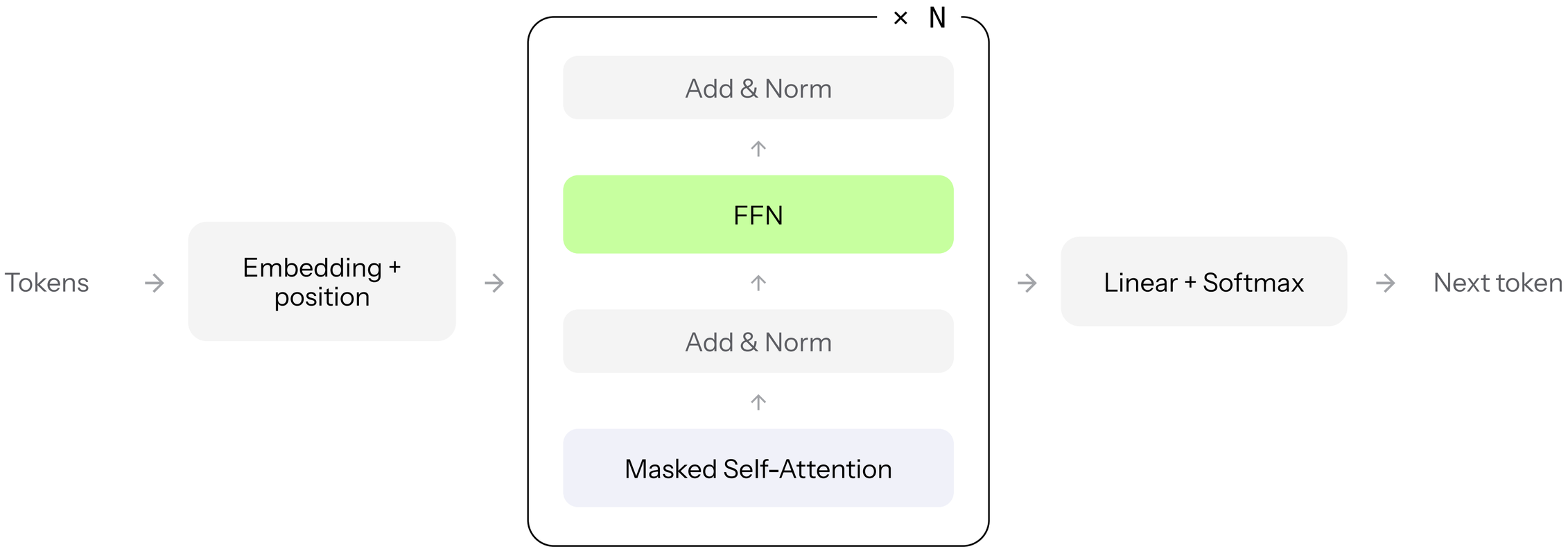
Cost grows linearly with parameters



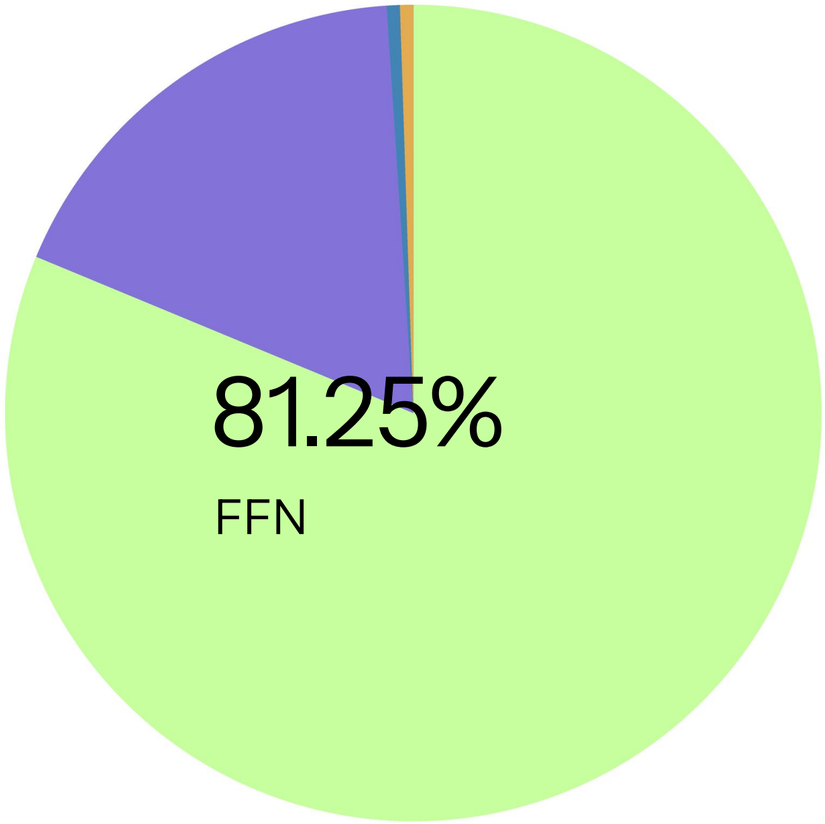
MoE cost does not grow linearly with parameters



Transformer architecture

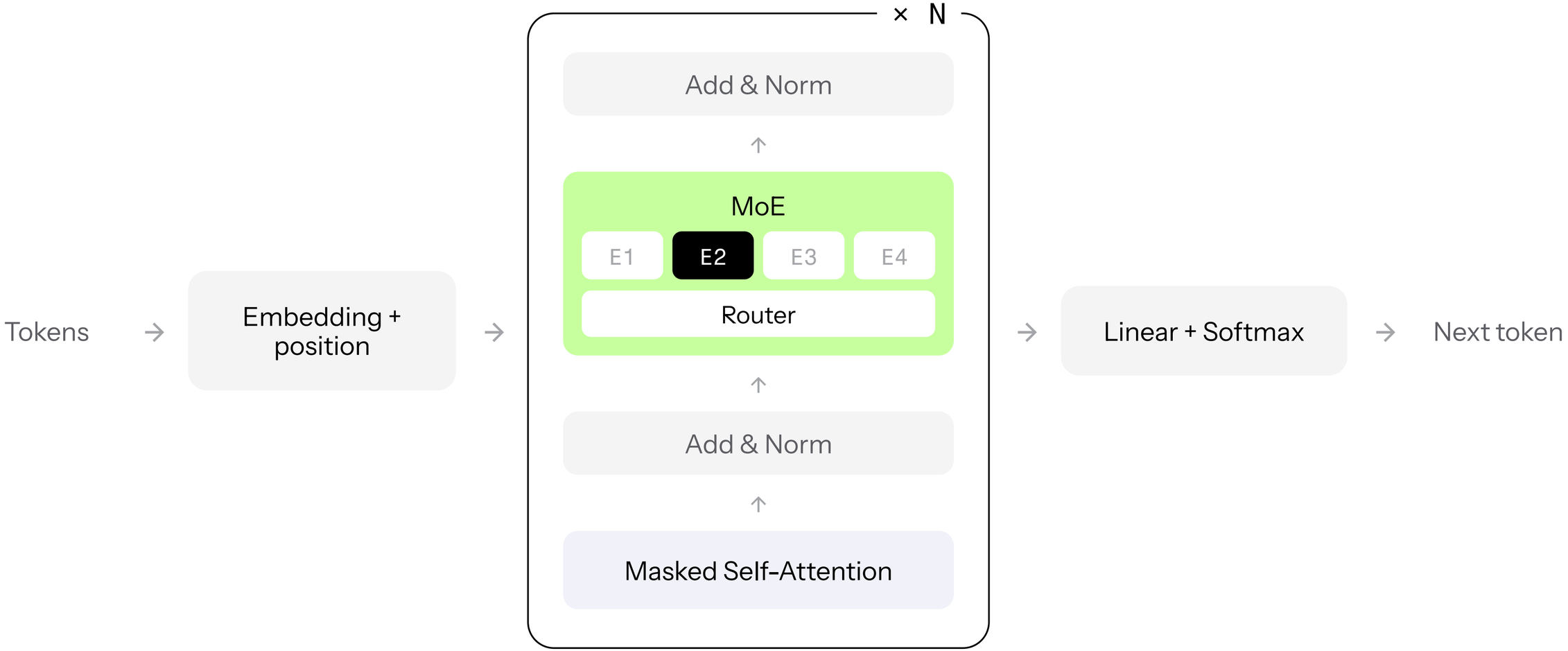


Llama 3.1 405B: where are the parameters?



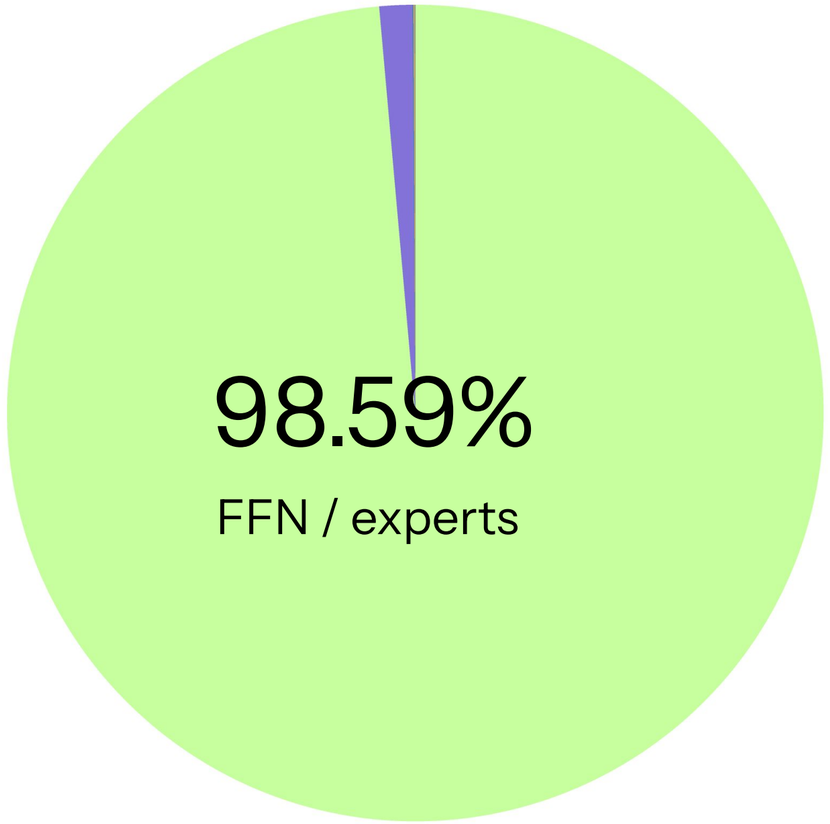
FFN	81.25%
Attention (GQA)	17.71%
Input embeddings	0.52%
Output head	0.52%
Norms	<0.01%

Transformer architecture with MoE



Kimi K3: where are the parameters?

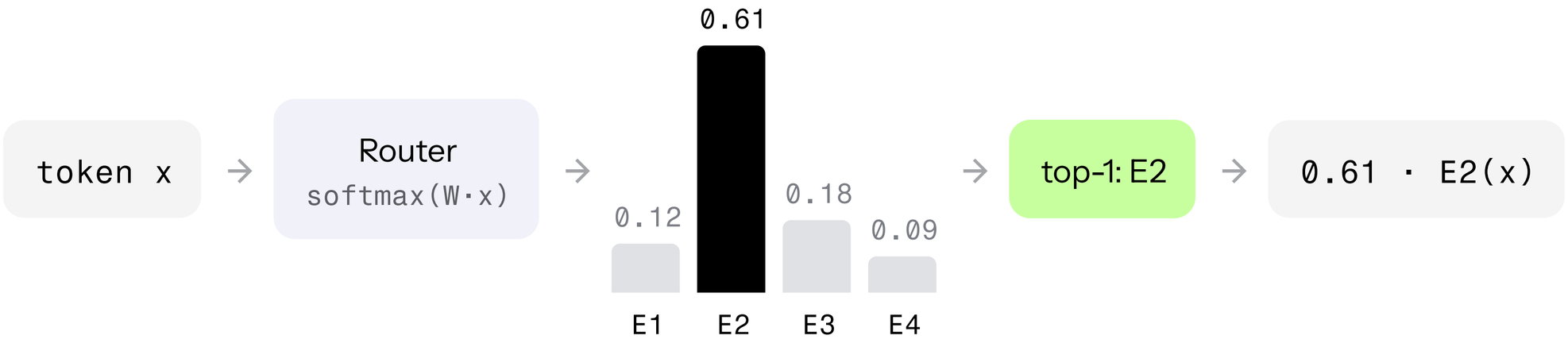
~2.8T language parameters · all experts counted



FFN / experts	98.59%
Attention (KDA + MLA)	1.30%
Input embeddings	0.04%
Output head	0.04%
Routers, norms, residuals	0.02%

Top-1

Switch Transformer



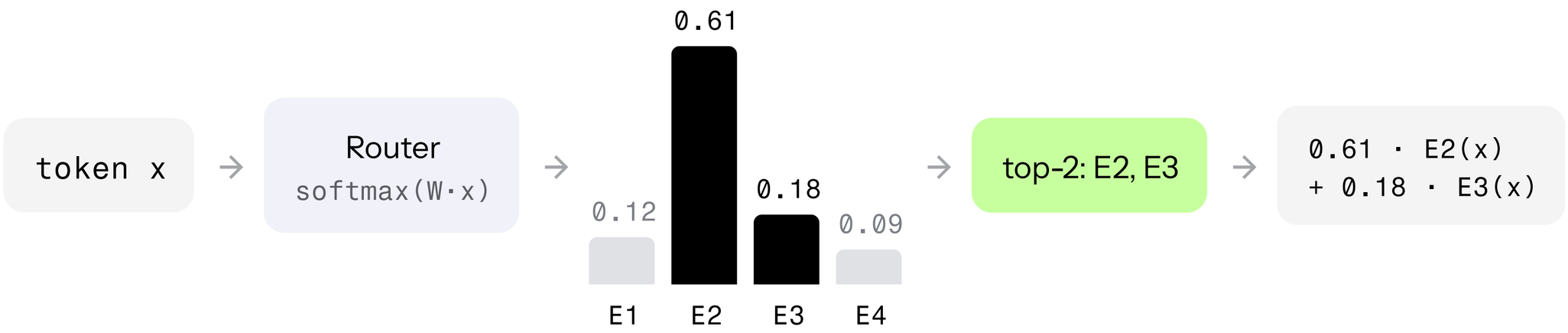
Why multiply by p?

```
probs = softmax(router(x))
i = argmax(probs)              # int, no gradient
p = probs[i]

y = expert[i](x)               # router.grad = 0
y = p * expert[i](x)           # grad flows through p
```

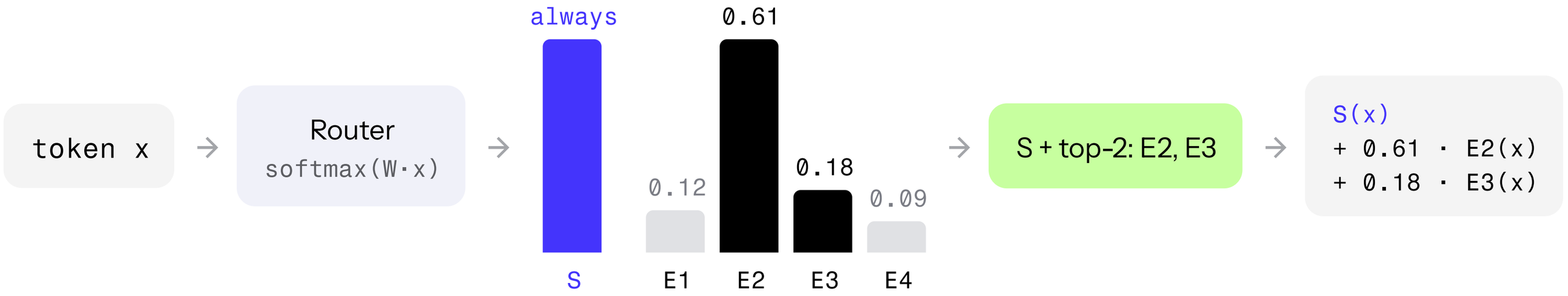
Top-k

GShard, Mixtral, DeepSeek



Shared expert

DeepSeekMoE, DeepSeek-V3



Hours of training to reach the same quality

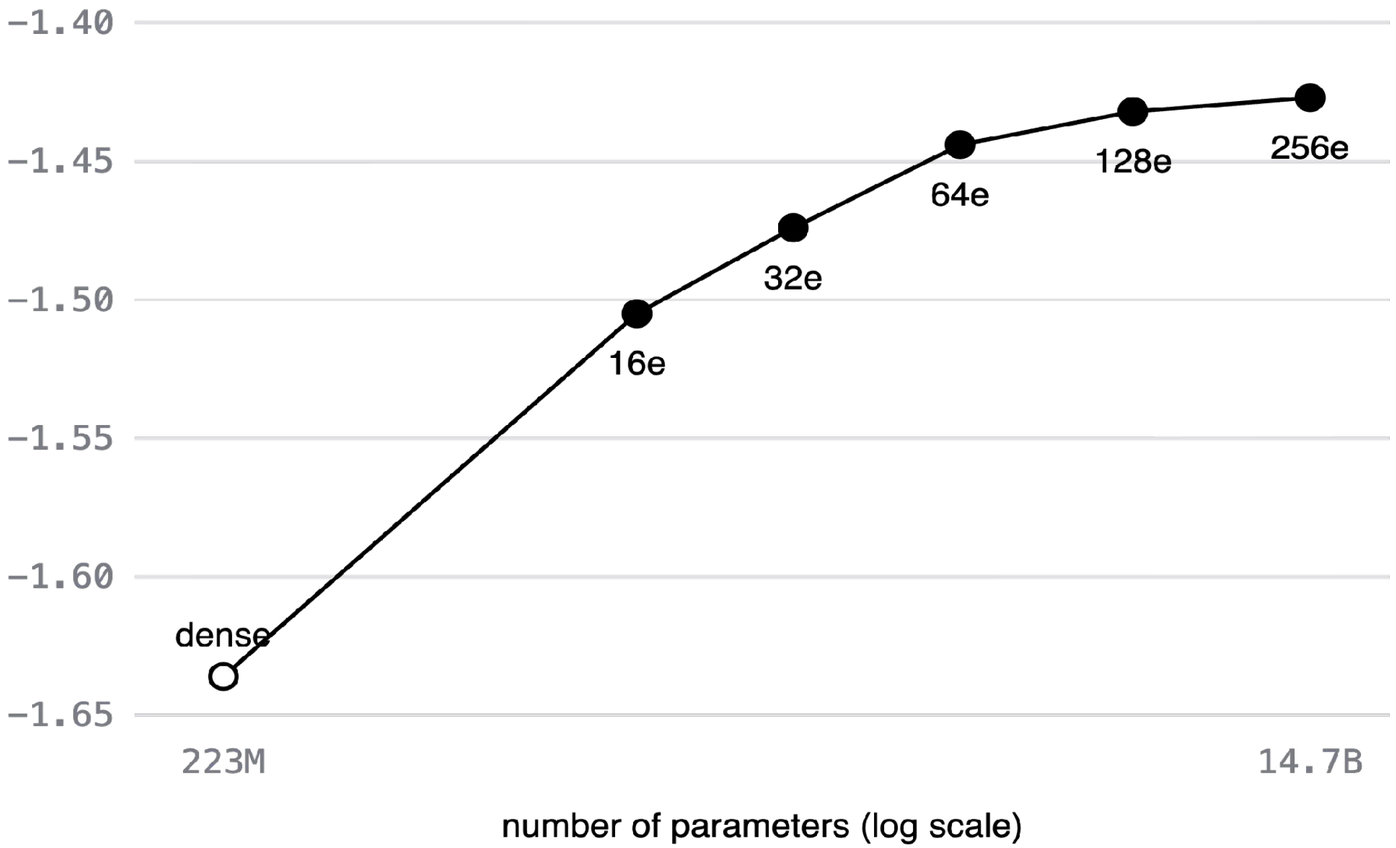
same hardware · Switch-Base has the same cost per token as T5-Base



Switch Transformers (Fedus et al., 2022), Table 1

Results: more experts

same FLOP/token · neg. log perplexity (↑)



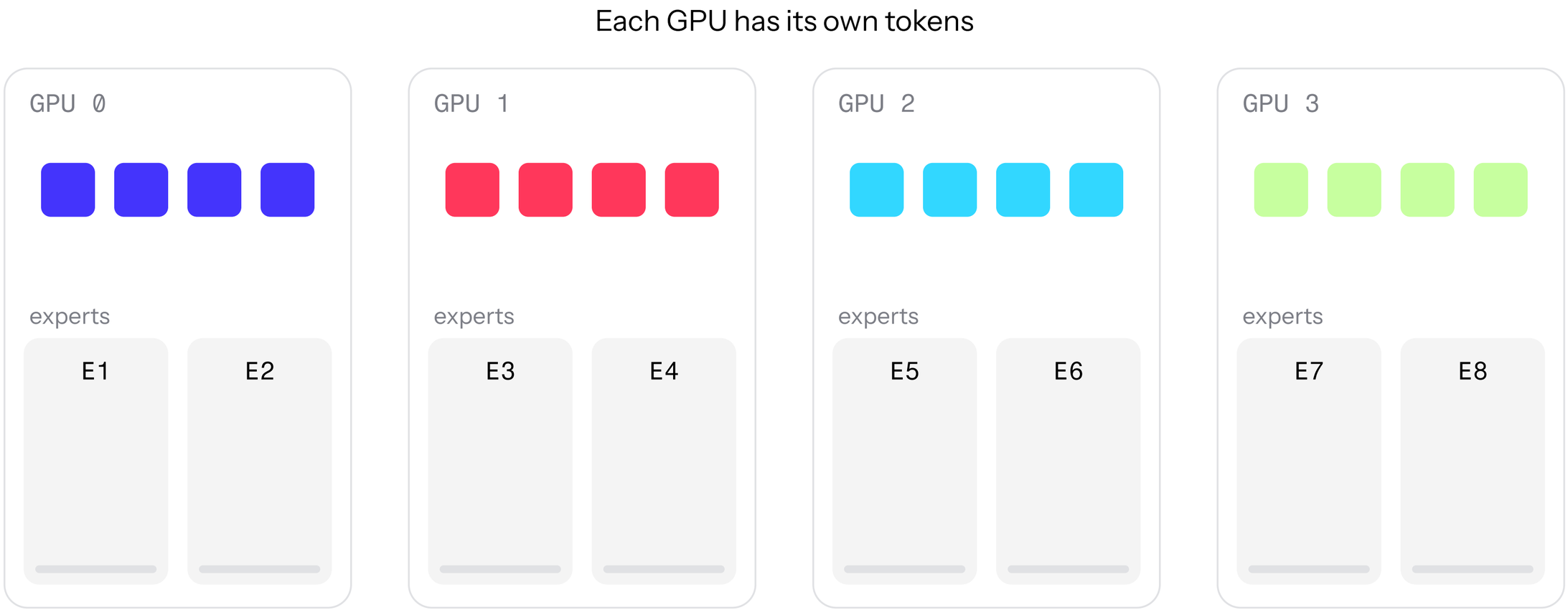
Switch Transformers (Fedus et al., 2022), Table 7

What about memory?

	Dense 671B	MoE 671B	Dense 37B
Weights in memory (FP8)	~671 GB	~671 GB	~37 GB
FLOPs per token	~1.3 TFLOP	~74 GFLOP	~74 GFLOP

Expert parallelism

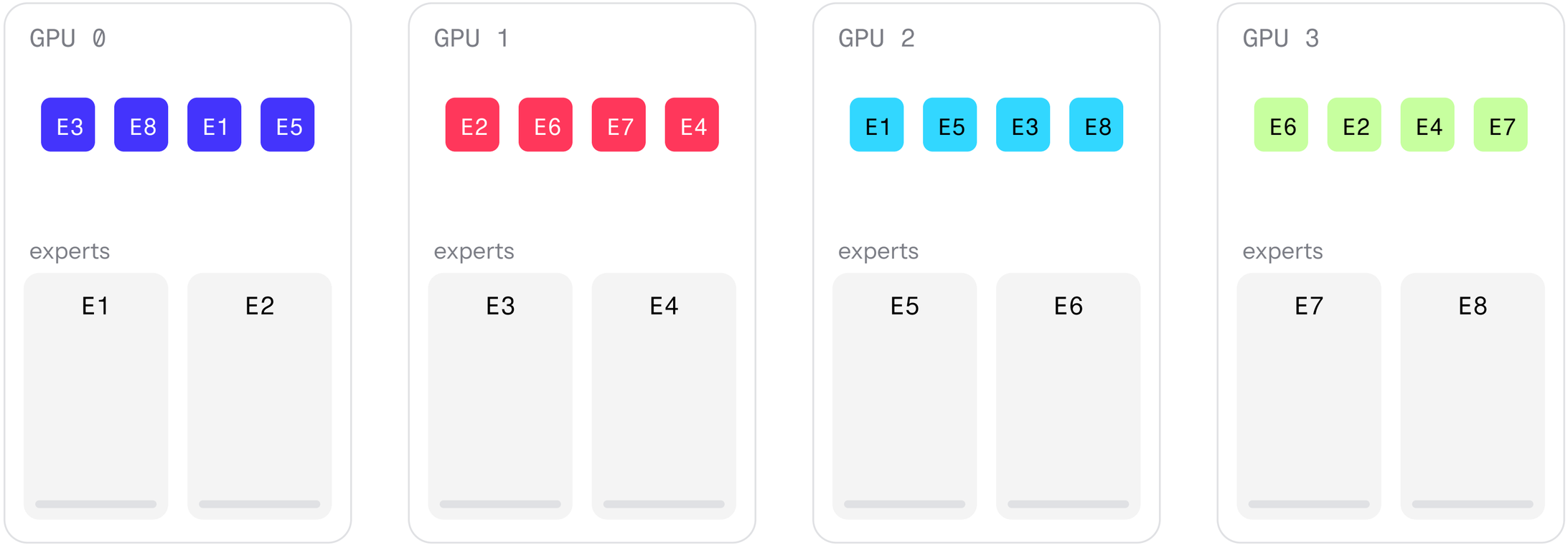
experts spread across GPUs · two all-to-alls per layer



Expert parallelism

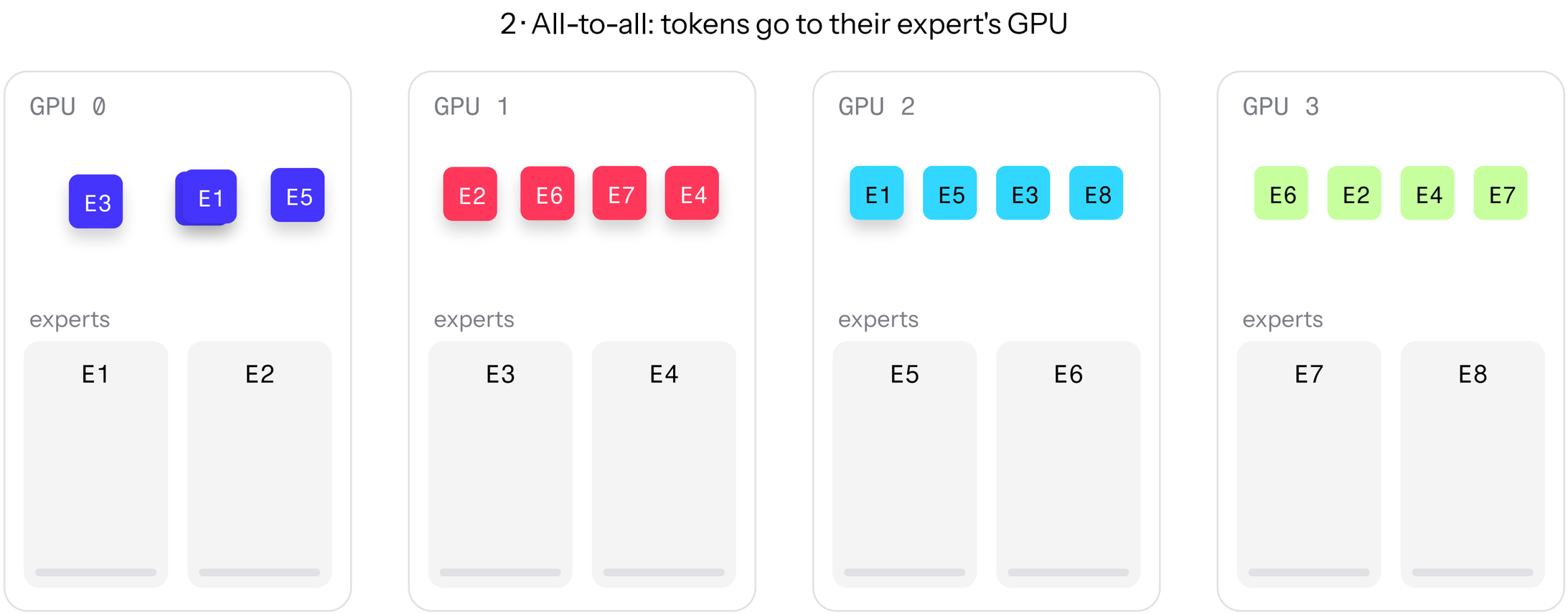
experts spread across GPUs · two all-to-alls per layer

1 · Router picks an expert



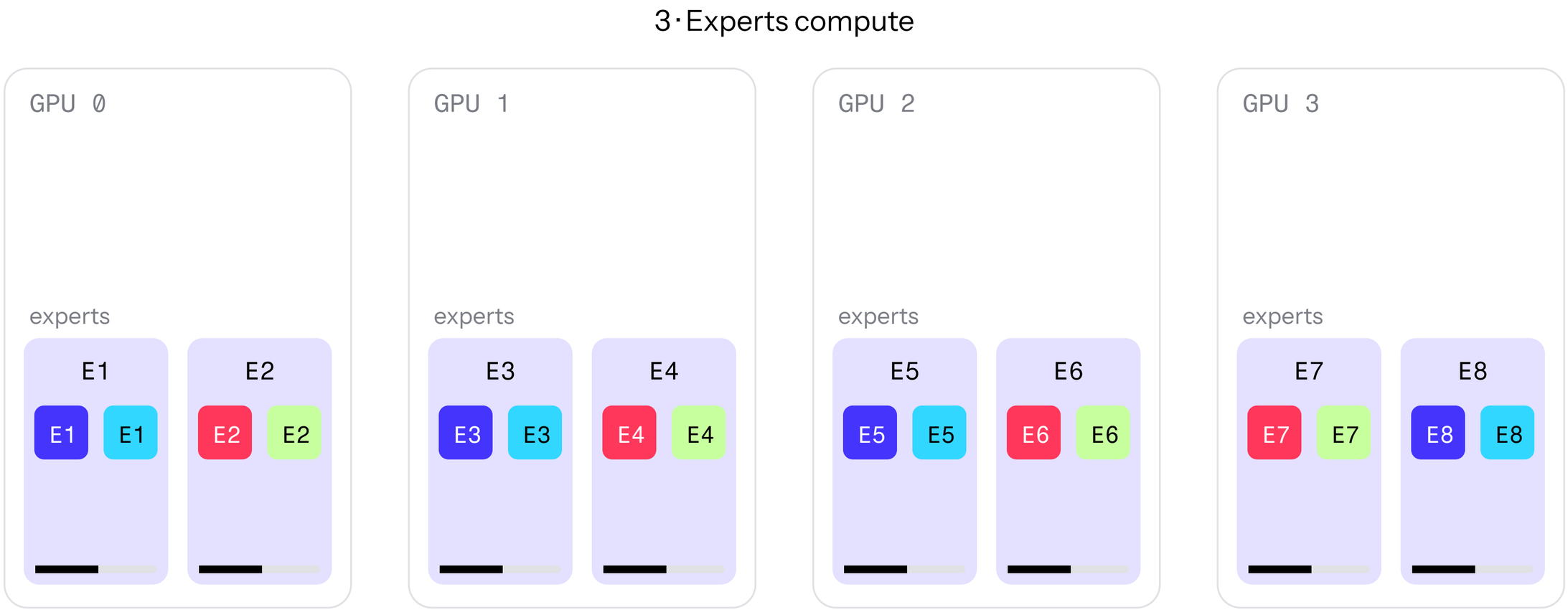
Expert parallelism

experts spread across GPUs · two all-to-alls per layer



Expert parallelism

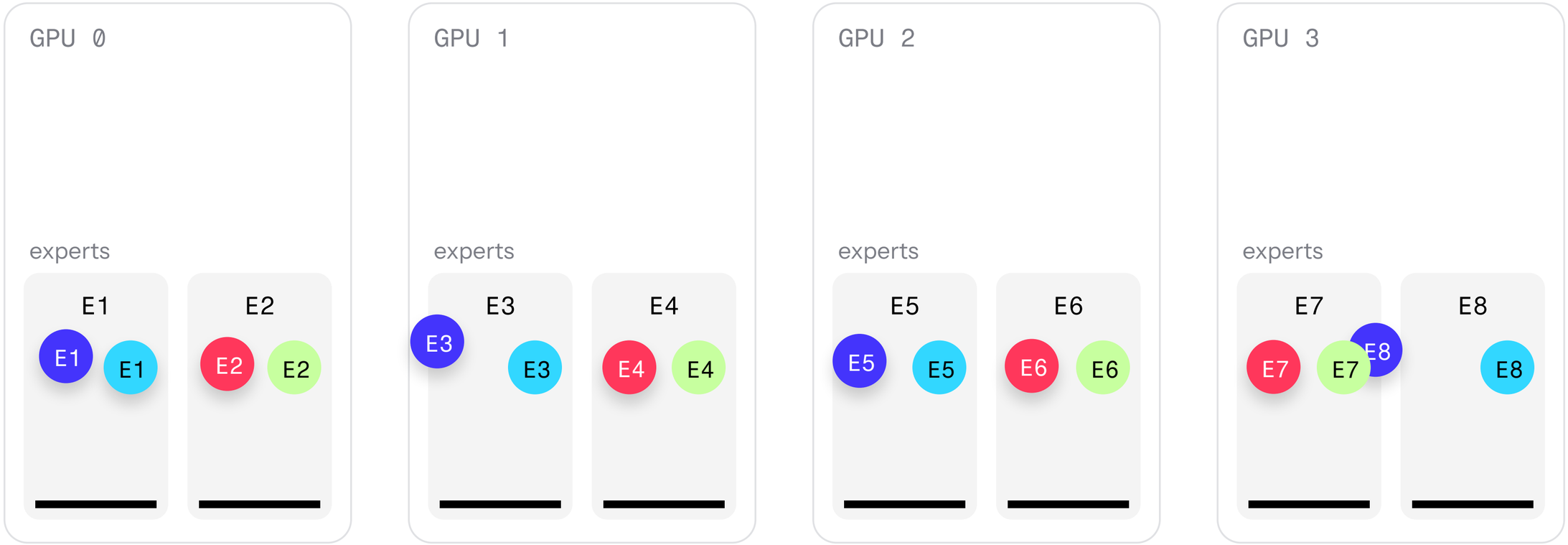
experts spread across GPUs · two all-to-alls per layer



Expert parallelism

experts spread across GPUs · two all-to-alls per layer

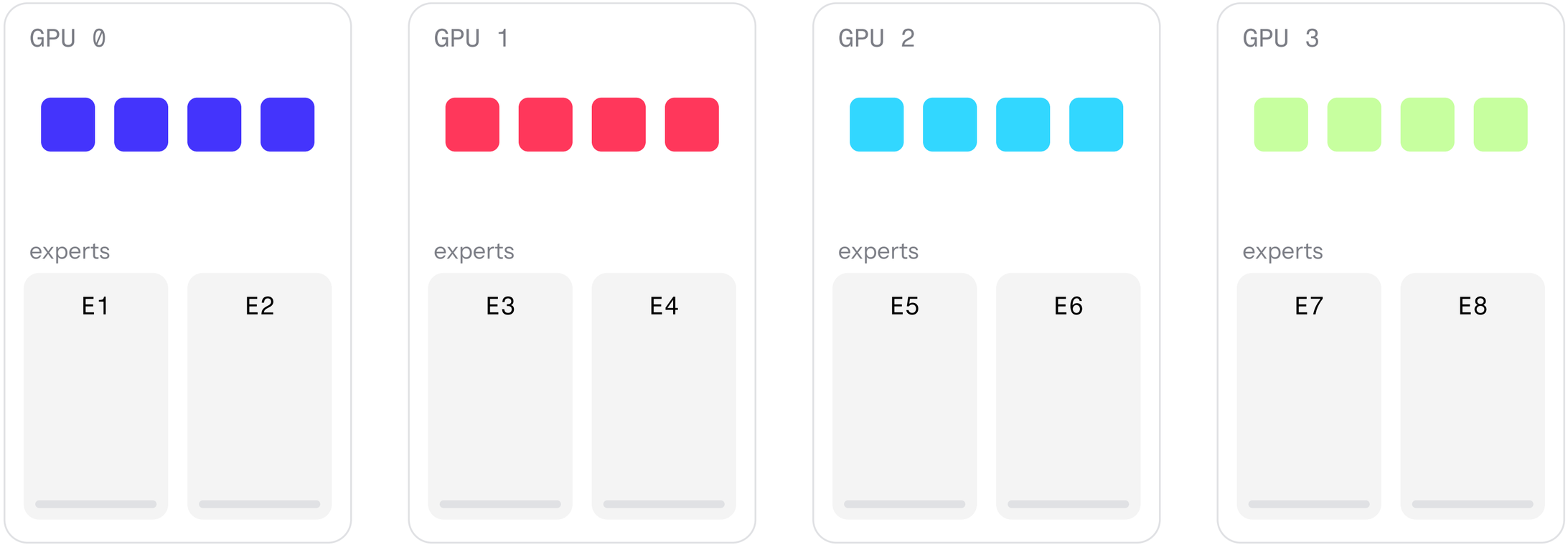
4 · All-to-all: results return



Expert parallelism: overload

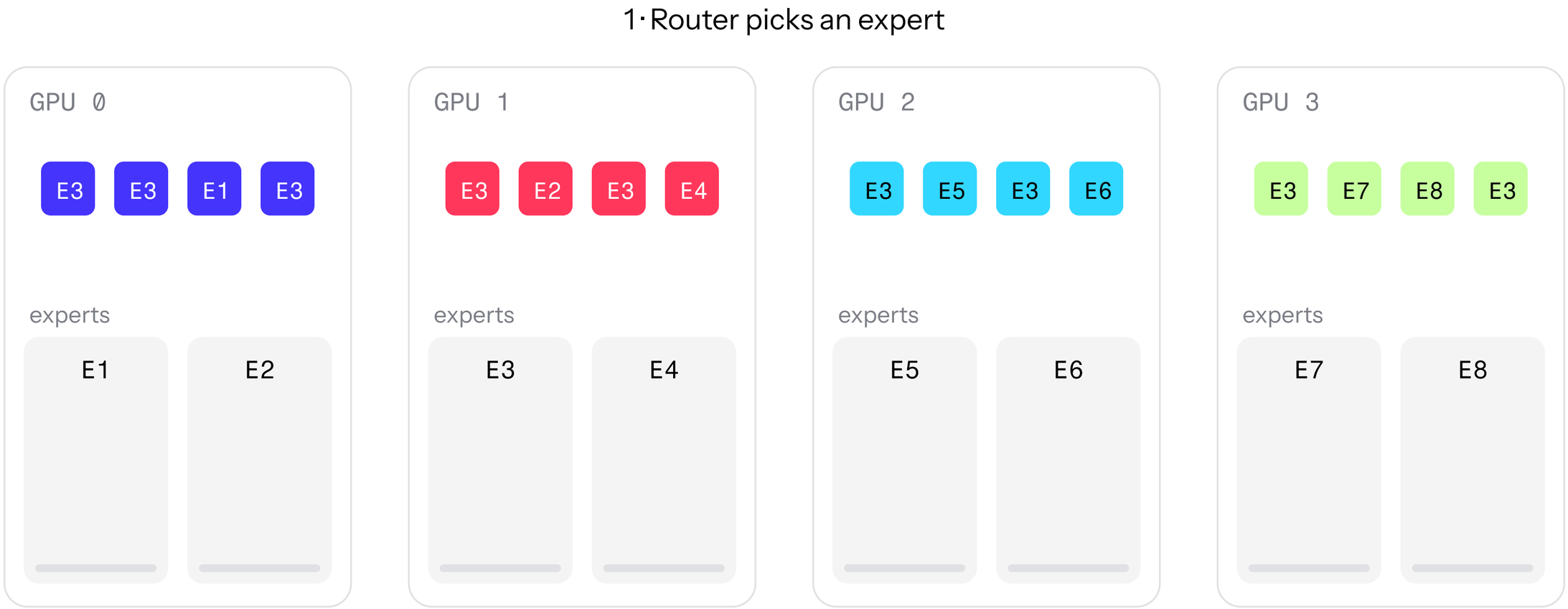
the whole step waits for the slowest GPU

Each GPU has its own tokens



Expert parallelism: overload

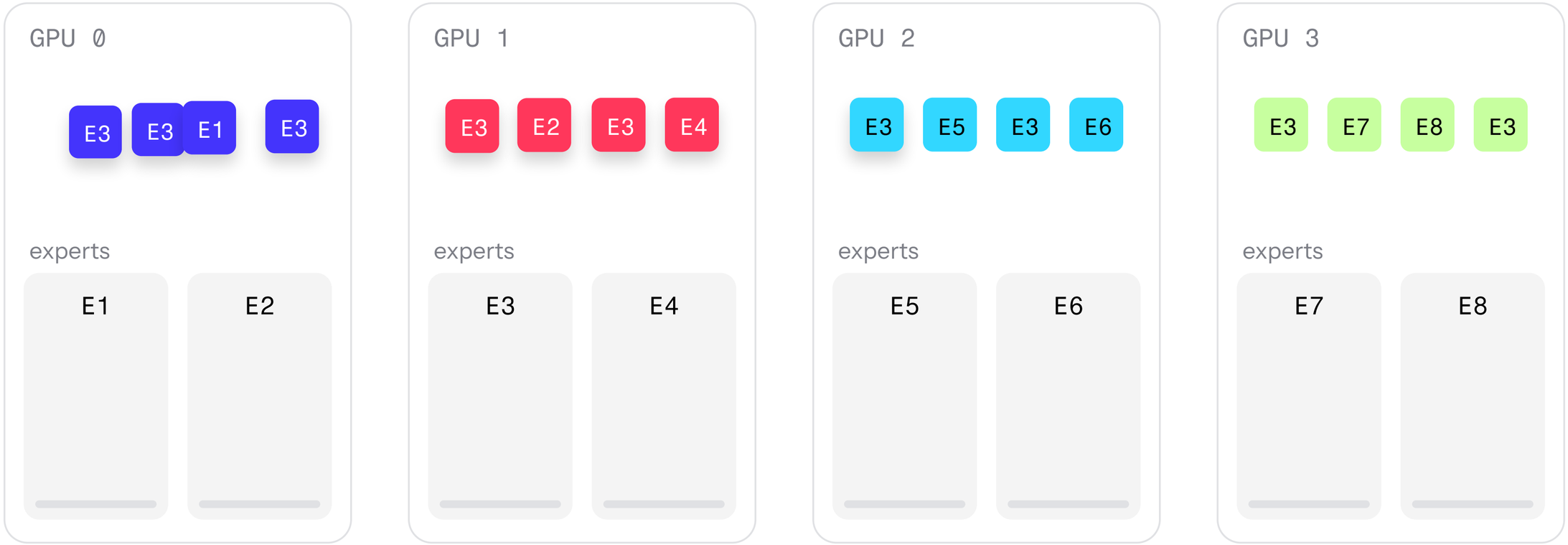
the whole step waits for the slowest GPU



Expert parallelism: overload

the whole step waits for the slowest GPU

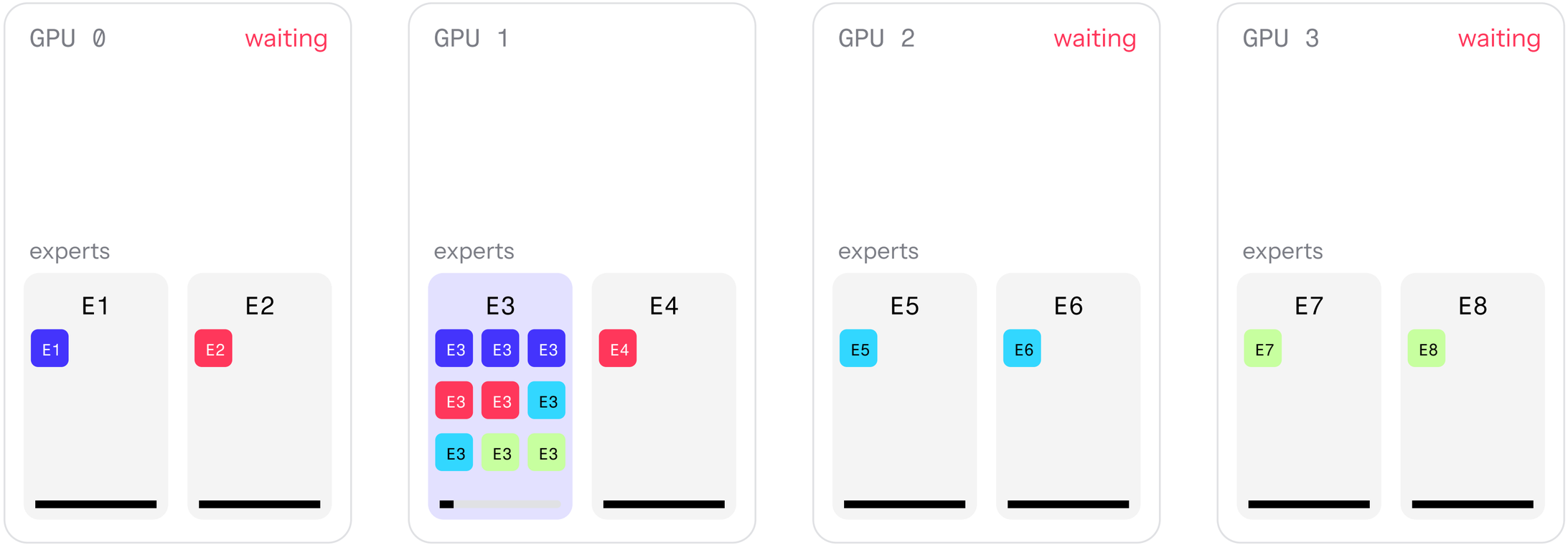
2 · All-to-all: tokens go to their expert's GPU



Expert parallelism: overload

the whole step waits for the slowest GPU

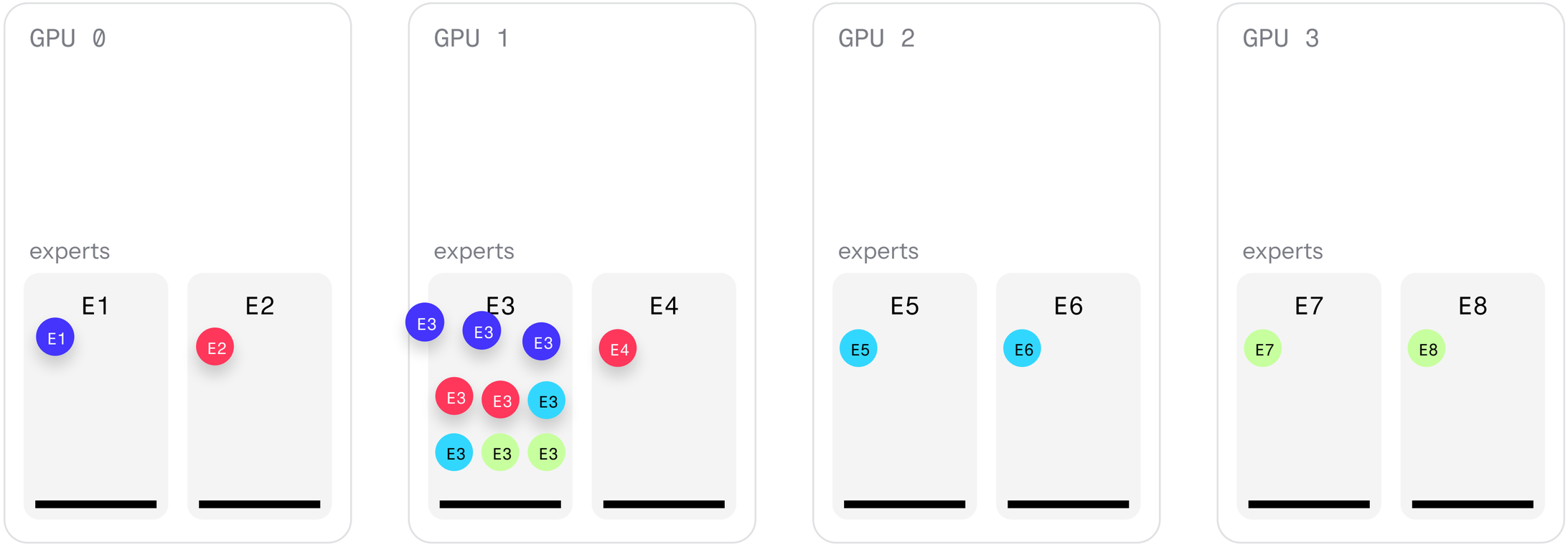
3 · E3 processes 9 tokens, the other GPUs wait



Expert parallelism: overload

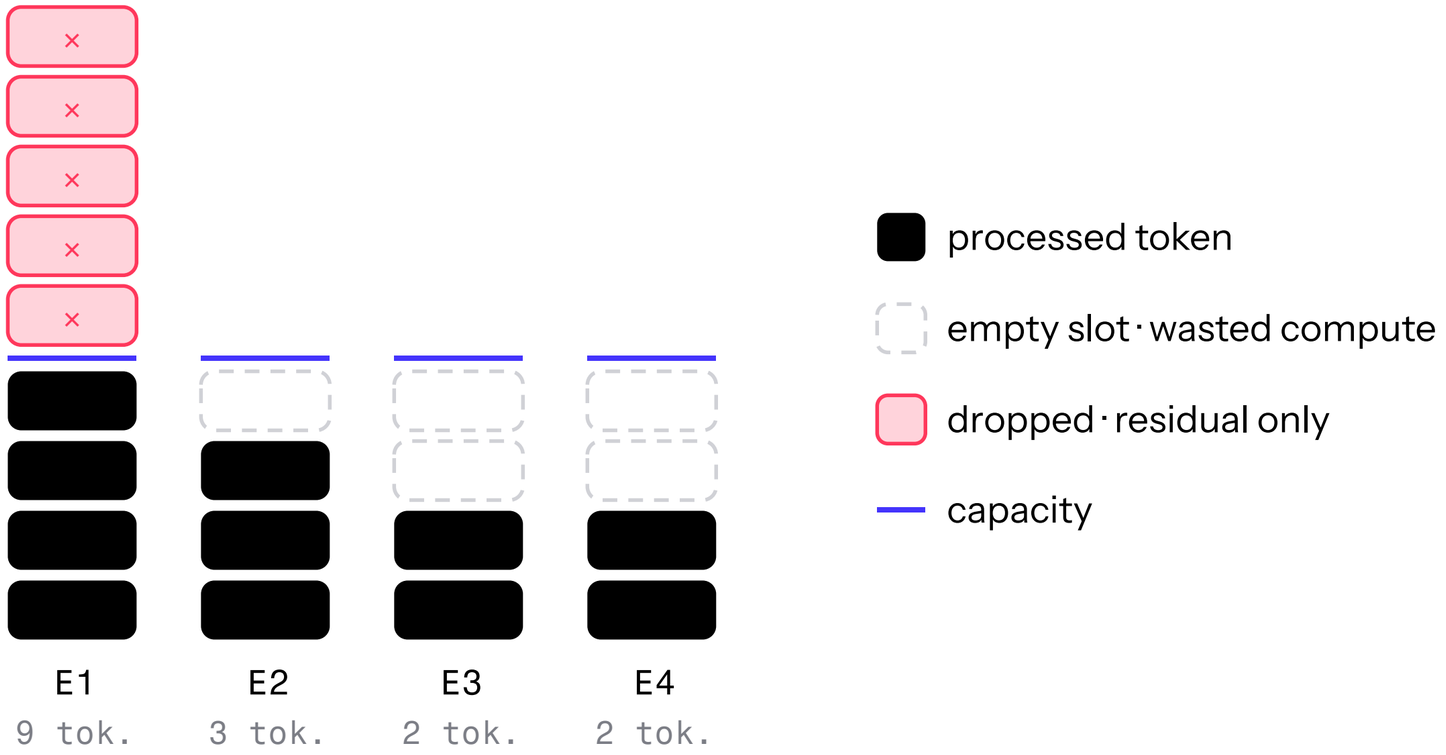
the whole step waits for the slowest GPU

4 · All-to-all: results return



Expert capacity

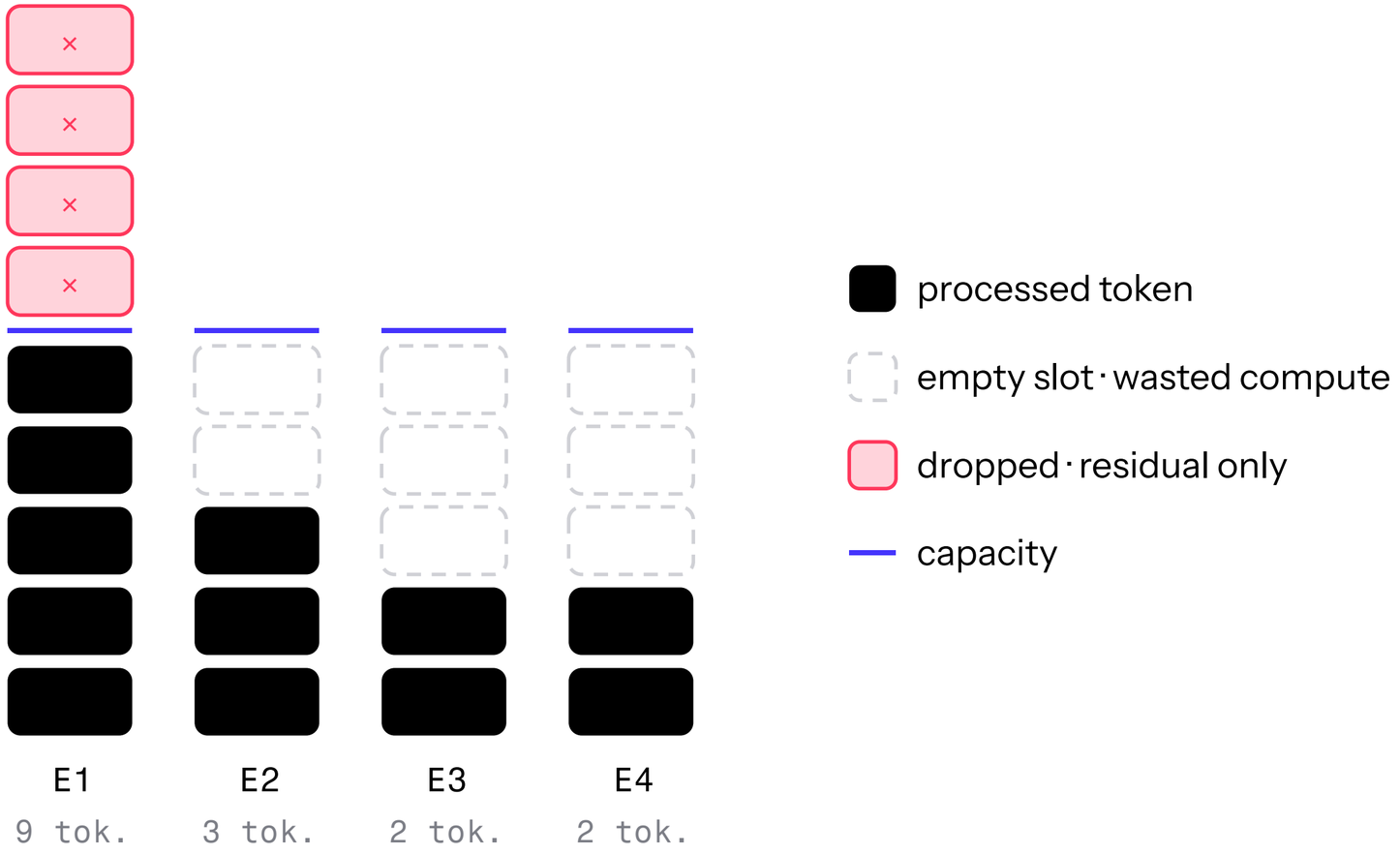
`capacity = tokens / experts × CF = 16 / 4 × 1.0 = 4`



5 dropped 5 empty slots

Expert capacity

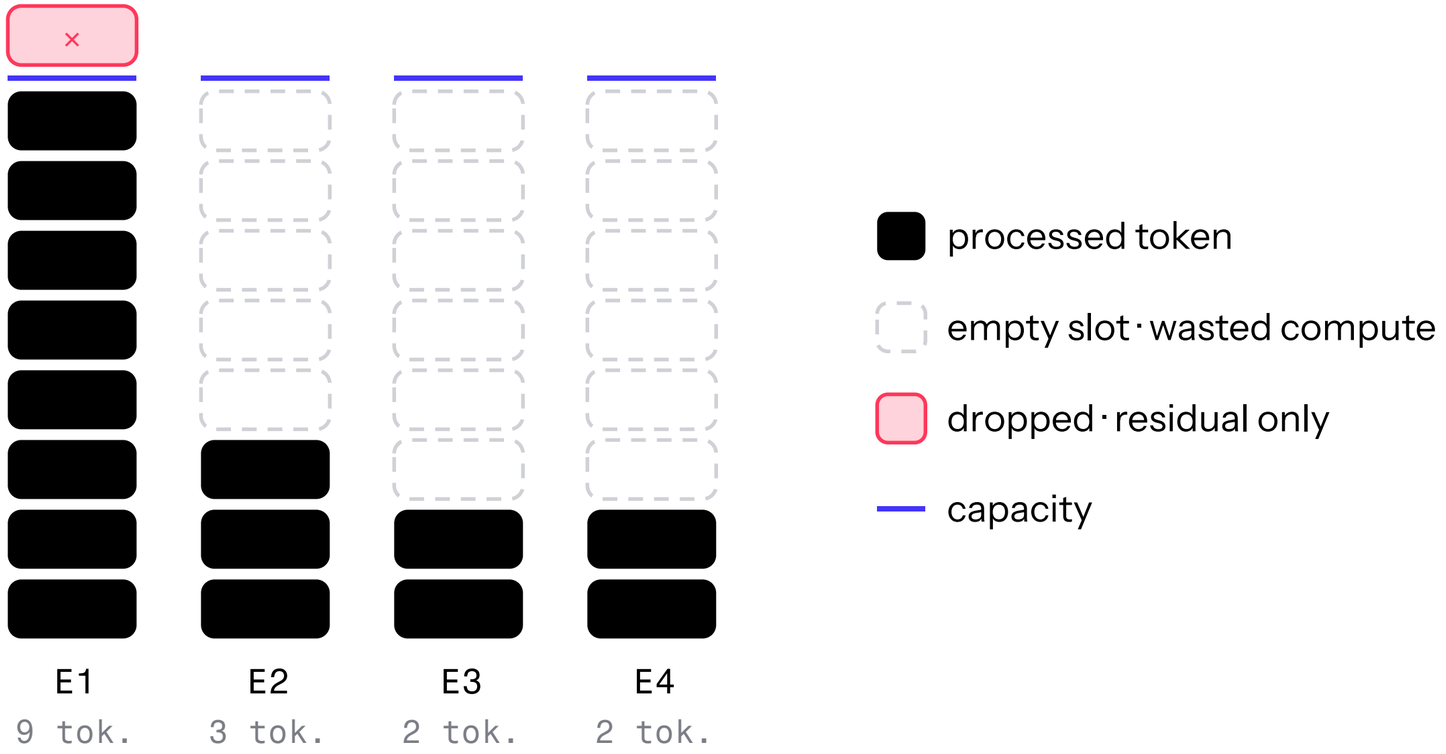
`capacity = tokens / experts × CF = 16 / 4 × 1.25 = 5`



4 dropped 8 empty slots

Expert capacity

`capacity = tokens / experts × CF = 16 / 4 × 2.0 = 8`



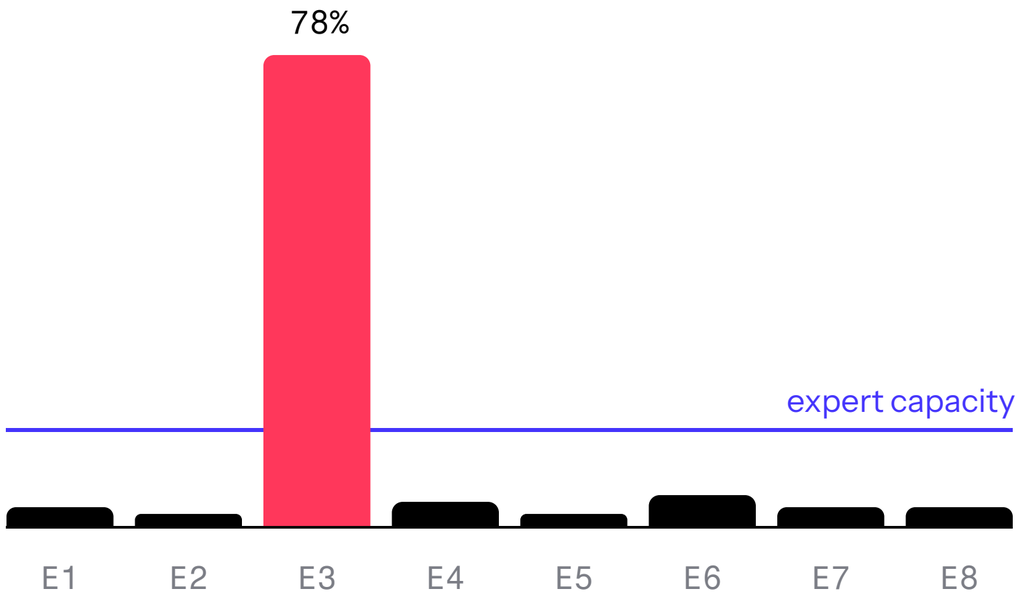
1 dropped 17 empty slots

Routing collapse

Start of training



After collapse



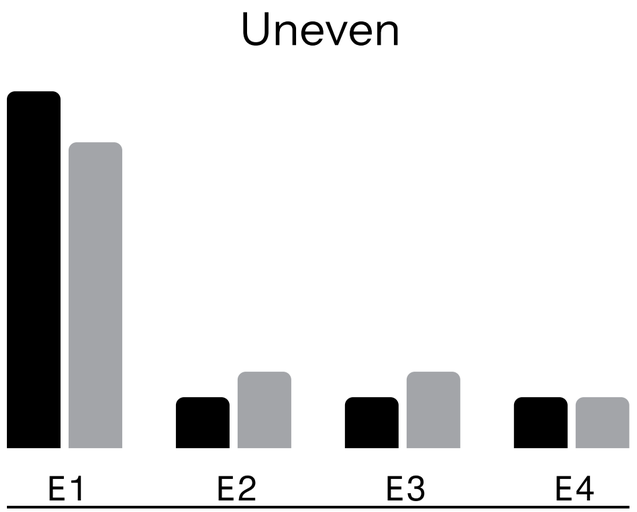
expert gets more tokens → learns faster → router picks it more often ↻

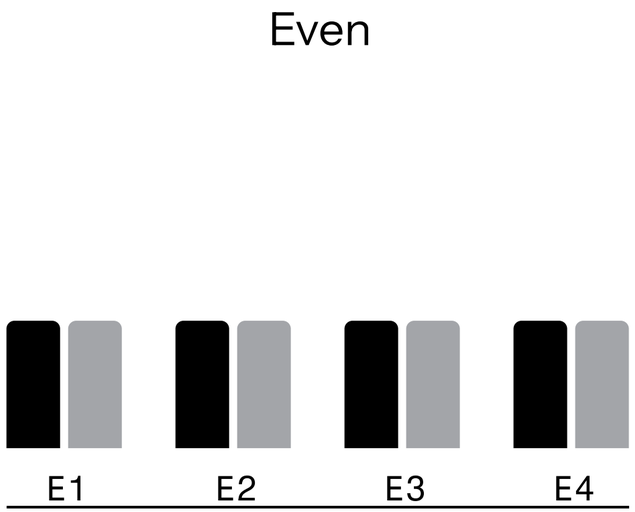
Auxiliary load balancing loss

$$L = L_{\text{task}} + \alpha L_{\text{aux}}$$

$$L_{\text{aux}} = N \sum_{i=1}^{N} f_i P_i$$

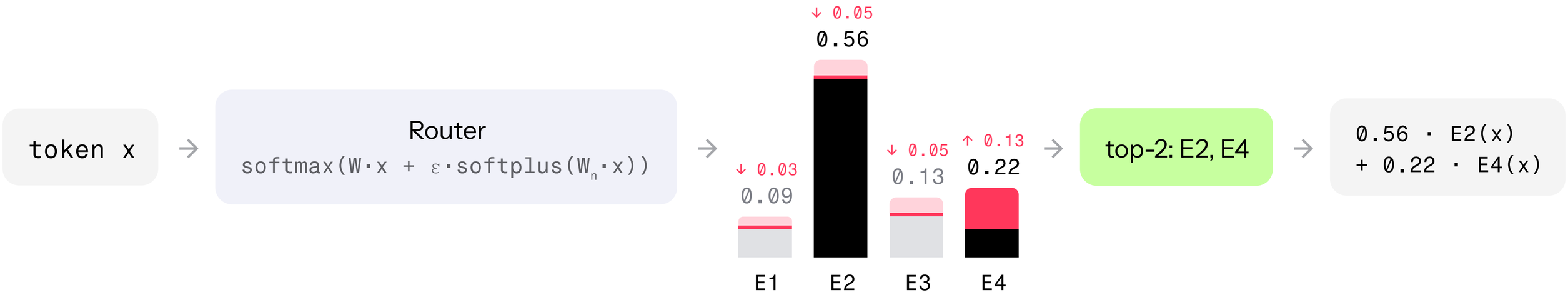
- f_i — fraction of tokens sent to expert i
- P_i — mean router probability for expert i
- N — number of experts
- α — weight, e.g. 0.01





Noisy top-k

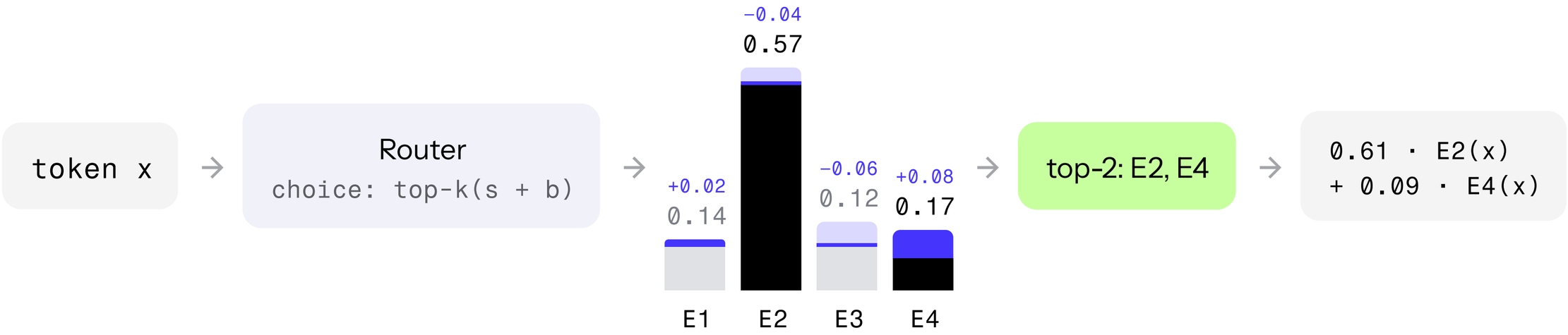
Shazeer et al. 2017



noise changes the choice: E4 instead of E3

Bias instead of aux loss

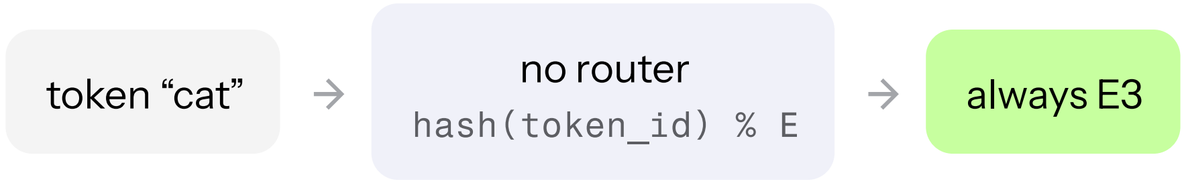
DeepSeek-V3



bias changes the choice, not the weights

Hash routing

Fixed expert per token · Roller et al. 2021



01

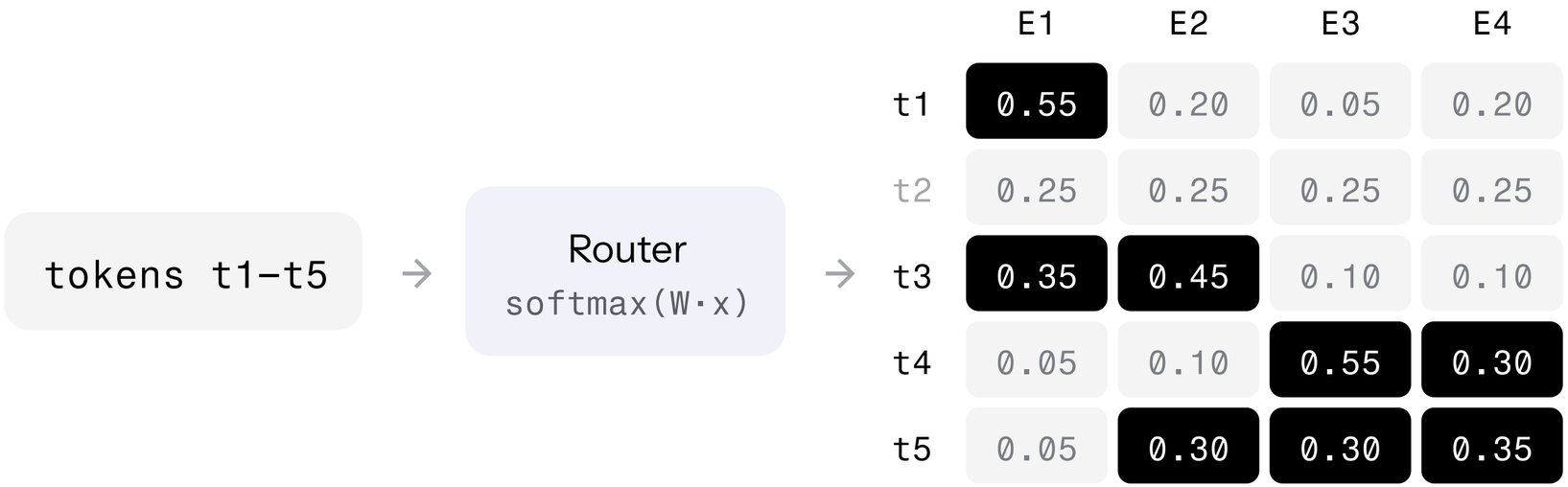
Token ID picks the expert.
The expert still sees context.

02

More parameters, one active FFN.
No learned router.

Expert choice

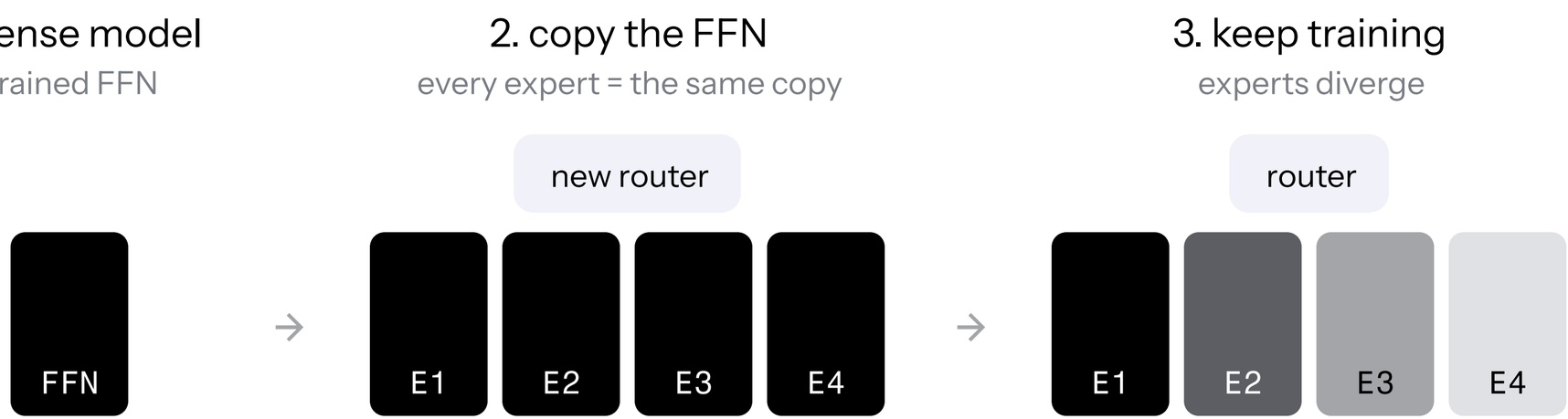
experts choose tokens



each expert takes its top-2 tokens

Upcycling

MoE from a trained dense model instead of from scratch · Komatsuzaki et al. 2022



Thank you

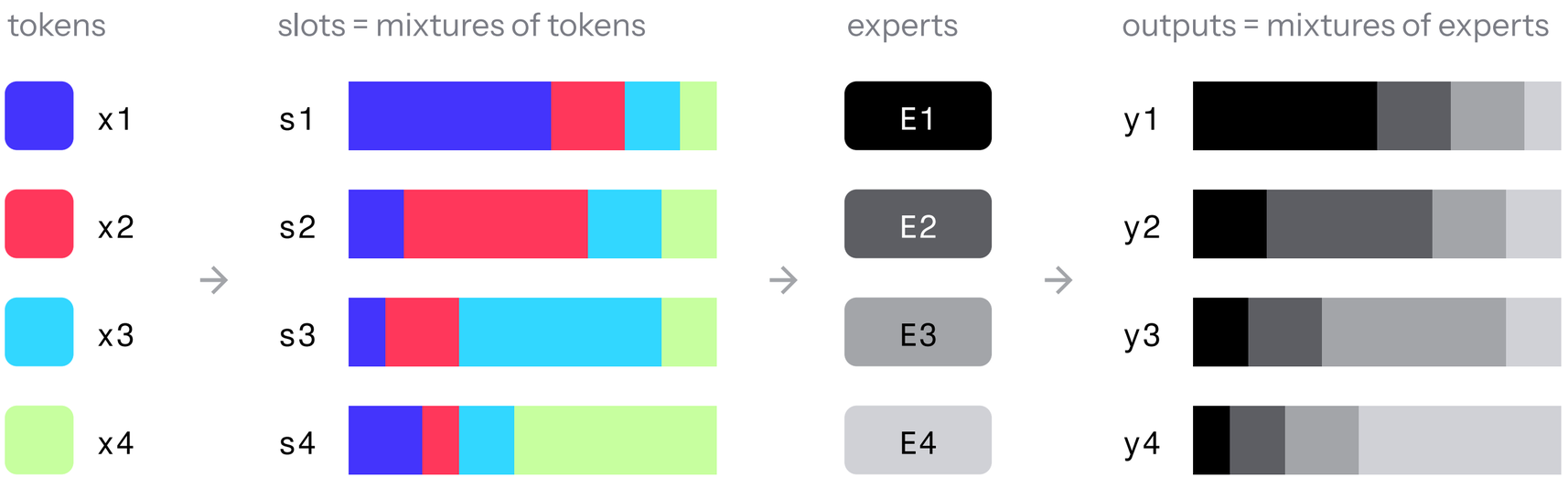
Sources

Vaswani et al., 2017	Attention Is All You Need	arXiv:1706.03762
Shazeer et al., 2017	Outrageously Large Neural Networks: The Sparsely-Gated MoE Layer	arXiv:1701.06538
Kaplan et al., 2020	Scaling Laws for Neural Language Models	arXiv:2001.08361
Lepikhin et al., 2020	GShard	arXiv:2006.16668
Fedus et al., 2022	Switch Transformers	arXiv:2101.03961
Roller et al., 2021	Hash Layers For Large Sparse Models	arXiv:2106.04426
Zhou et al., 2022	Mixture-of-Experts with Expert Choice Routing	arXiv:2202.09368
Komatsuzaki et al., 2022	Sparse Upcycling	arXiv:2212.05055
Puigcerver et al., 2023	From Sparse to Soft Mixtures of Experts	arXiv:2308.00951
Jiang et al., 2024	Mixtral of Experts	arXiv:2401.04088
Dai et al., 2024	DeepSeekMoE	arXiv:2401.06066
Wang et al., 2024	Auxiliary-Loss-Free Load Balancing for MoE	arXiv:2408.15664
DeepSeek-AI, 2024	DeepSeek-V3 Technical Report	arXiv:2412.19437
DeepSeek-AI, 2026	DeepSeek-V4	arXiv:2606.19348
DeepSeek-AI, Z.ai	DeepSeek and GLM technical reports and model cards	

Bonus slides

Soft MoE

experts receive weighted mixtures of tokens



Results: a trillion parameters

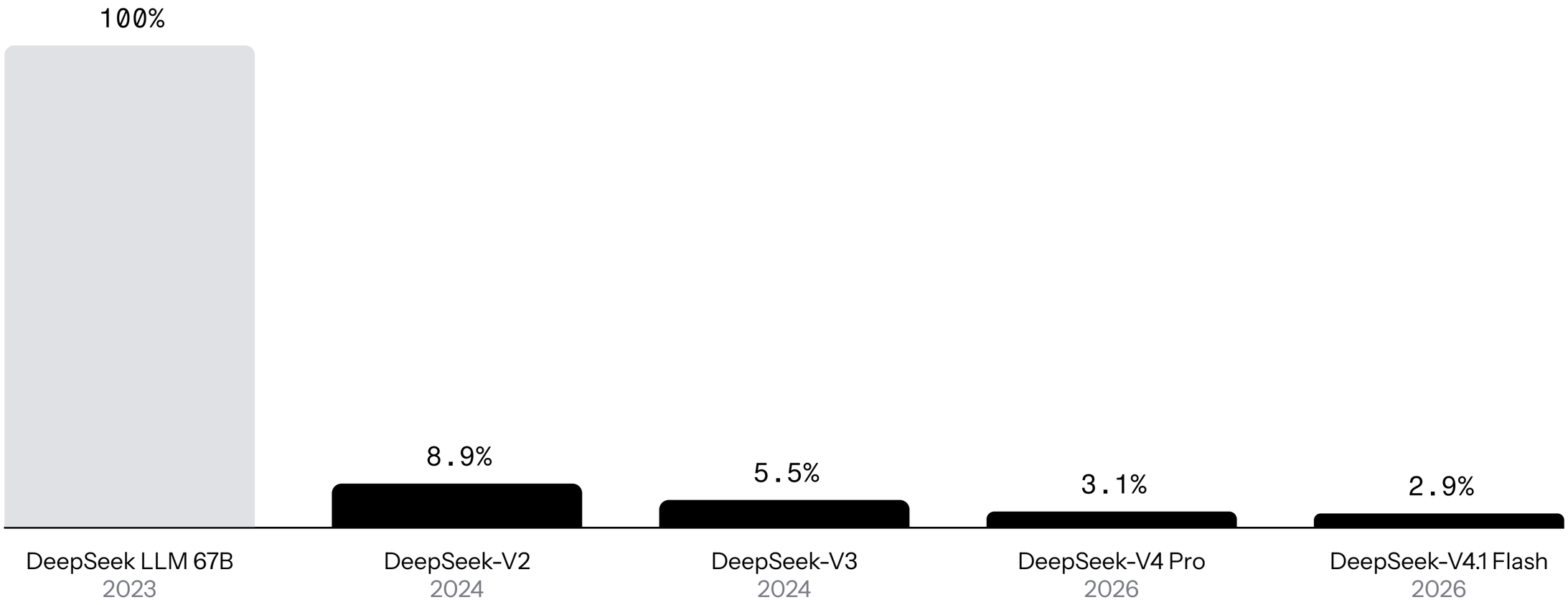
after 500k pre-training steps

	T5-XXL (dense)	Switch-C (2048 experts)
Parameters	11B	1571B
FLOPs / sequence	6.3T	890B
Neg. log perplexity (↑)	−1.095	−1.043

Switch Transformers (Fedus et al., 2022), Table 9

DeepSeek

% of parameters used per token dense MoE



GLM

% of parameters used per token dense MoE

